

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ ТА ІНФОРМАТИКИ

Кафедра теорії оптимальних
процесів

АЛГОРИТМИ І СТРУКТУРИ ДАНИХ У СА
Лабораторна робота №1
“ Сорткування і порядок зростання”

Виконала:

Студентка групи ПМа-21

Лаврик А.С.

Перевірили:

ас. кафедри ТОП Зяць А. Р.

Львів – 2026

Мета роботи:

Побачити, як три різні алгоритми сортування поведуться на однакових даних.

Умова завдання:

- 1) Реалізувати три алгоритми сортування, названі у варіанті, не викликаючи стандартних функцій сортування.
- 2) Підготувати дев'ять масивів цілих чисел: по три на 100, 1 000 і 10 000 елементів — у випадковому, впорядкованому та зворотному порядку.
- 3) Відсортувати кожен масив кожним із трьох алгоритмів і виміряти час, кількість порівнянь і кількість перестановок.
- 4) Звести числа в таблицю й назвати порядок зростання, який з неї видно для кожного алгоритму.

Варіант 9:

набір ключів: журнал відвідувань спортивного залу

алгоритм 1: сортування Шелла

алгоритм 2: гномове сортування

алгоритм 3: спрощений Timsort

Результат виконання програми:

SORTING ALGORITHMS COMPARISON

Algorithm	Data	Size	Comparisons	Swaps	Time

Shell	Random	100	861	399	0.024 ms
Gnome	Random	100	4934	2419	0.140 ms
TimSort	Random	100	999	931	0.108 ms
Shell	Sorted	100	503	0	0.010 ms
Gnome	Sorted	100	99	0	0.002 ms
TimSort	Sorted	100	224	0	0.064 ms
Shell	Reverse	100	668	260	0.016 ms
Gnome	Reverse	100	9900	4950	0.281 ms
TimSort	Reverse	100	1566	1694	0.095 ms
Shell	Random	1000	15321	7784	0.375 ms
Gnome	Random	1000	480621	239815	16.000 ms
TimSort	Random	1000	13496	12644	1.593 ms
Shell	Sorted	1000	8006	0	0.159 ms
Gnome	Sorted	1000	999	0	0.020 ms

TimSort	Sorted	1000	3528	0	1.681 ms
Shell	Reverse	1000	11716	4700	0.262 ms
Gnome	Reverse	1000	999000	499500	30.959 ms
TimSort	Reverse	1000	17844	20404	1.314 ms
Shell	Random	10000	270684	155793	7.974 ms
Gnome	Random	10000	50062641	25026323	1554.420 ms
TimSort	Random	10000	172708	164017	18.547 ms
Shell	Sorted	10000	120005	0	3.203 ms
Gnome	Sorted	10000	9999	0	0.195 ms
TimSort	Sorted	10000	56407	0	16.999 ms
Shell	Reverse	10000	172578	62560	4.049 ms
Gnome	Reverse	10000	99990000	49995000	3443.012 ms
TimSort	Reverse	10000	194488	241208	19.747 ms

Github: <https://github.com/annalavryk/algorithm/tree/Lab1>

Висновки:

Ми розглядали три методи сортування: сортування Шелла, гномове сортування, спрощений Timsort. Сортування Шелла-це вдосконалений варіант сортування вставками. Головна ідея: Спочатку сортуємо елементи далеко один від одного, поступово зменшуємо відстань і в кінці сортуємо сусідні елементи. Гномове сортування-це коли ми порівнюємо сусідні елементи. Якщо вони неправильні-міняємо місцями й повертаємося назад, якщо правильні-рухаємось вперед. Timsort-це гібридне сортування. Основна ідея: не сортувати масив тупо з нуля, а знайти частини, які вже більш-менш впорядковані і розумно їх об'єднати. Тобто ми маємо знайти вже впорядковані частини масиву, за потреби трохи їх відсортувати, а потім ефективно об'єднати ці частини.

На випадкових масивах зі збільшенням кількості елементів різниця між алгоритмами стає більш помітною.

Гномове сортування виконує значно більше переміщень, оскільки для перестановки елементів йому часто доводиться повертатися на попередні позиції. Сортування Шелла працює ефективніше завдяки використанню великих кроків, які дозволяють швидше переміщувати елементи до потрібних позицій.

Спрощений Timsort також показує хорошу ефективність, оскільки використовує сортування вставками для невеликих частин і подальше їх злиття.

Для вже впорядкованих масивів алгоритми виконують значно менше переміщень. Гномове сортування у цьому випадку може працювати дуже швидко, оскільки сусідні елементи вже знаходяться у правильному порядку. Timsort також добре працює з такими даними, оскільки окремі частини масиву вже фактично є впорядкованими.

Найгірша ситуація для гномового сортування виникає тоді, коли масив впорядкований у зворотному напрямку. Для кожного елемента необхідно виконати багато перестановок, тому зі збільшенням розміру масиву час роботи швидко зростає.

Сортування Шелла та Timsort значно краще справляються з великими масивами такого типу.